Query Performance Prediction Is Not Benefit Prediction: Observability for Selective Retrieval Interventions

Anonymous Author(s)

Abstract

LLM query expansion (QE), cross-encoder reranking, and pseudo-relevance feedback (PRF) each help some queries and harm others, which motivates applying them selectively. A selective controller needs two predictions: whether retrieval is failing (*detection*, a query performance prediction task) and whether a specific intervention will help or harm the query (*action utility*, a benefit prediction). We measure both on the same queries from the system's own *observability*, the qrel-free telemetry it records about its retrieval, on TripClick and BEIR Natural Questions: LLM QE under BM25 and a dense retriever, and reranking and RM3-style PRF under BM25. Detection AUC is 0.62–0.70, and under BM25 no better than a standard unsupervised predictor (NQC). Predicting harm reaches only 0.50–0.61; detection beats it, paired-significantly, in seven of eight settings, though the gap is partly structural, since an intervention rarely changes a failing query: a failure detector is not a substitute for a benefit predictor. Failure is also the wrong target for a policy. A perfect failure detector would recover only 8–57% of an oracle selector's gain, depending on intervention and corpus; the rest lies in healthy queries. On NQ, gating reranking on predicted failure halves its gain (+0.114 vs. +0.225 nDCG@10); on TripClick, where reranking hurts on average, the failure gate still loses (−0.010) while a benefit model does not (−0.001). For LLM QE, failure-selective expansion halves generator calls and on NQ significantly reduces unconditional QE's harm, but no QE policy measurably beats BM25 alone.

1 Introduction

LLMs can rewrite a query into pseudo-documents, paraphrases, or disambiguating context, and conditioning the rewrite on retrieved evidence improves on query-only rewriting [13]. Applied unconditionally, though, LLM QE harms a measurable fraction of queries [1, 21]. Selective expansion is the natural response, and it needs two predictions:

$$\underbrace{P(\text{failure} \mid x)}_{\text{detection}} \qquad \text{vs.} \qquad \underbrace{P(\Delta\text{nDCG@10} < 0 \mid x, a)}_{\text{action utility (benefit prediction, as harm)}}$$

Detection asks whether the current ranking is already bad. Action utility asks whether a *specific* expansion a will make it worse; its mirror, whether a will help, matters as much for a policy (Section 5). That the two differ is not new: selective-QE work predicts expansion benefit directly [2, 7], and uplift modelling shows that targeting high-risk cases is not the same as targeting cases a treatment helps [3, 8]. What is missing is a measurement of detection and action utility on the same queries, and of what each one is worth to a policy. We take LLM QE as the main case and repeat the measurement for two classical interventions, cross-encoder reranking and PRF, to see which findings are specific to generative expansion. We use *observability* for qrel-free telemetry a search system records about its own behaviour (score distributions, coverage, ranking stability, reranker agreement), the signal a deployed controller can actually see.

Contributions. (C1) A paired, same-query measurement of detection and action-utility AUC in eight settings (three interventions, two corpora, BM25 and dense retrieval for LLM QE), with the label's tie rule, a standard QPP baseline, and a combined pre+post-action model. **(C2)** A headroom decomposition per intervention: how much of an oracle selector's gain perfect failure detection alone would recover (8–57%), and why the remainder needs action-utility prediction. **(C3)** A policy evaluation with minimum detectable effects, showing that gating on failure can halve a helpful intervention's gain and fail to stop a harmful one's loss.

2 Related Work

Selective QE and query performance prediction. Pre- and post-retrieval QPP (Clarity [6], WIG [24], NQC [18]) predicts query difficulty. Selective QE goes further: Cronen-Townsend et al. [7] decide whether to expand by comparing the clarity of the expanded and unexpanded rankings, i.e. they predict expansion benefit directly; Amati et al. [2] gate expansion on difficulty predictors; Lv and Zhai [14] predict a per-query feedback weight; and Collins-Thompson [5] trades expansion reward against risk. These works evaluate the gated system in aggregate; they do not report detection and benefit prediction as two AUCs on the same queries, nor decompose the headroom between them.

LLM QE and its failures. Query2doc [20], MuGI [23], and CSQE [13] expand unconditionally. Weller et al. [21] find generative expansions often hurt strong retrievers; Abe et al. [1] trace failures to unfamiliar or ambiguous queries. Kotte [10] gates prompt-only rewriting for dense retrievers (gate AUC 0.593, no reliable gain over never rewriting); Adaptive-RAG [9] routes on a question-complexity classifier. Neither separates the gate's detection accuracy from its benefit prediction.

Uplift modelling. Predicting an outcome is not predicting a treatment's effect on it [8]; targeting the customers most likely to churn can be ineffective [3]. Our setting has an advantage marketing lacks: both outcomes (expanded and unexpanded) are observed for every query offline, so the uplift label is exact up to relevance-label noise.

3 Setup

Data. TripClick [16]: 1.52M health documents; the TAIL query bucket's validation and test splits (1,175 queries each), with the click-derived relevance labels TripClick releases. BEIR NQ [11, 19]: 2.68M passages, human-annotated relevance; we carve a fixed-seed split (800 validation) and use a disjoint 2,052-query test set, run once under a scope committed before any of its data was touched. An earlier n=600 NQ pilot is reported only in the ledger below.

Pipeline. BM25 over SQLite FTS5 (Porter stemming, title weight selected on validation). From pre-action telemetry a deterministic rule picks one of four expansion operators (vocabulary, context, disambiguation, entity normalization) or none; it picks none for 1.5% (TripClick) and 1.9% (NQ) of queries under BM25, and 10.6% and 5.8% under dense retrieval. The rest form the *acted-upon* set, on which every AUC in Table 1 is computed. The generator (`qwen2.5:3b`, 4-bit, greedy, ≤ 2 calls per query) writes a corpus-grounded expansion from the top retrieved evidence; the expanded query is retrieved and, on TripClick and in both dense settings, its top candidates are reranked with a MiniLM cross-encoder [17]. The original ranking is never reranked, so outside NQ BM25 (no reranking on either side) the expansion effect includes the reranker's; Section 4 checks this. The dense setting replaces first-stage retrieval with `bge-small-en-v1.5` [22] over the full collections.

Labels. *Failure*: the first-stage top 10 contains no relevant document (nDCG@10 $= 0$); 45% of TripClick and 49% of NQ test queries under BM25. *Action utility*: Δ = nDCG@10(expanded) − nDCG@10(original), with *harmed* ($\Delta < 0$) as the positive class and ties ($\Delta = 0$) counted as not harmed. Ties are 48% (TripClick) and 69% (NQ) of acted-upon queries, and harm 33% and 17%. Harm is impossible on a failing query, since its nDCG is already 0.

Models and features. Detection: logistic regression on 8 pre-action features (top score, score margin, score entropy, query-term coverage, mean rare-term IDF, entity overlap, result coherence, ranking stability under a small query perturbation); 6 under dense retrieval (no IDF, no stability). Action utility: 10 post-action features (gains in score margin, coverage and coherence; stability; rewrite drift; reranker–first-stage disagreement; top-k overlap with the original; reranker top score and margin; query-length ratio), fit by linear regression on Δ under BM25 and as a logistic harm classifier under dense retrieval (9 features, no stability). Every model and threshold is fit on validation and scored once on test; significance is a paired bootstrap over queries (10^4 resamples, seed 42).

Policies. *BM25*: never expand. *Static QE*: always expand; TripClick deploys a reciprocal-rank fusion of the original and expanded rankings (no fused ranking was stored for NQ, whose arms use the expanded ranking itself). *Failure-selective QE*: expand iff $P(\text{fail} \mid x) > \tau$, τ the median validation prediction. *Closed-loop QE* (TripClick only): expand, then accept the fused ranking, roll back, or re-plan once based on the action-utility score, with validation-calibrated thresholds.

Two further interventions. Under a separate pre-committed protocol, we apply two classical interventions to every query of the BM25 run. *Reranking*: the MiniLM cross-encoder scores the BM25 top 100; its top 50 lead, and the BM25 order follows. *PRF*: RM3-style [12] term weights from the top d documents (softmax-normalized BM25 score times term frequency) select t expansion terms, whose BM25 ranking is interpolated with the original after min-max normalization, with weight λ on the original. (d, t, λ) is chosen per corpus on validation over a 108-setting grid: $(20, 10, 0.9)$ on TripClick, $(20, 30, 0.6)$ on NQ. Labels and detectors are those above; the action-utility model is a linear regression on Δ over the 8 pre-action features plus 6 label-free post-action ones (top-10 overlap, top-1 change, and rank-biased overlap with BM25; the new ranking's top score, margin, and score entropy), fit on validation. Every test query receives the intervention, so these AUCs use the full test sets.

BM25 baseline. Our first, unstemmed BM25 scored nDCG@10 0.228 on TripClick TAIL test, below the published 0.267 [16]. Re-selected on validation only (validation 0.235 → 0.327), BM25 alone (no reranker) scores 0.315 on test.

Test-access ledger. TripClick TAIL test was used during development before our data-use protocol existed: an early verifier and its thresholds were calibrated on it, two apparatus defects were found there (the reranked order was not wired into the accepted ranking; the verifier judged a different ranking than the one accepted), and the weak 0.228 BM25 was observed there. We then froze the protocol, refit every model and threshold on validation, checked BM25 once (0.315), and ran one frozen evaluation that produced every TripClick test episode used here, plus a 150-query dense pilot and one full dense run. All later analyses, including this revision's, recombine those frozen records under pre-specified rules without refitting on test. TripClick is therefore partly exploratory; NQ, untouched until its scope was committed, is the confirmation. NQ test was touched three times, each under a written pre-commitment: the n=600 pilot, disjoint from the 2,052 test queries (detection 0.638 $[0.593, 0.681]$, action utility 0.522 $[0.442, 0.602]$), the confirmatory 2,052-query run, and its dense rerun; two later BM25 re-runs on the same queries recomputed telemetry and score lists only. The reranking and PRF runs touched each test set once more, with every setting frozen on validation beforehand. Scripts and pre-commitments are in the anonymized repository: https://anonymous.4open.science/r/agentic-query-expansion-8939/.

4 Result 1: Detection vs. Action Utility

Under the pre-committed harm label, detection beats action utility on the same queries in three of four settings (Table 1). On the full test populations, detection AUC is 0.649, 0.680, 0.638, and 0.688. Dense TripClick is the exception; there the bi-encoder is weaker than BM25 (nDCG@10 0.217 vs. 0.315), while on NQ it is stronger (0.500 vs. 0.308).

Detection is ordinary QPP. The unsupervised NQC predictor over the top-100 BM25 scores [18] (normalized by the top-k mean, as FTS5 exposes no corpus score) matches our detector on TripClick (0.647 vs. 0.649; paired difference +0.003 $[-0.030, +0.035]$) and beats it on NQ (0.721 vs. 0.680; −0.041 $[-0.064, -0.017]$). WIG [24] is weaker (0.588 and 0.651). Detection is not the novel part; it is what selective QE already has.

Not a feature-set artifact. A logistic harm model on pre-action *plus* post-action features (18 under BM25, 15 dense) still trails detection on TripClick BM25 (0.603; difference +0.040 $[+0.003, +0.078]$) and dense NQ (0.625; +0.070 $[+0.033, +0.107]$), and does not differ from it on dense TripClick (0.609; +0.012 $[-0.034, +0.058]$). Adding pre-action features helps on dense NQ (+0.028 $[+0.010, +0.045]$ over post-action alone) but not on TripClick (BM25 +0.020 $[-0.013, +0.053]$; dense +0.008). NQ BM25 is missing: its test run stored no post-action features. A logistic refit of the BM25 action-utility model gives 0.563 (+0.011 $[-0.008, +0.029]$ over the linear one).

Not a reranker artifact. On TripClick BM25 the reranker alone lowers nDCG@10 (0.285 vs. 0.315), so the label mixes expansion with reranking. The frozen test run also holds the original query's

Table 1: Detection vs. action-utility (harm) AUC on each setting's acted-upon queries, with 95% CIs. "Diff." is the paired bootstrap of their difference on the same queries, so it equals the difference of the two columns. The last two columns use only queries whose nDCG@10 the expansion changed ($\Delta \neq 0$): action utility as helped-vs.-harmed, and the paired difference between that and detection (post hoc, not pre-committed; the two AUCs score different labels on the same queries).

Setting	n	Detection	Action utility	Diff.	Direction, $\Delta \neq 0$	Diff., $\Delta \neq 0$
TripClick, BM25	1157	0.643 [0.612, 0.674]	0.552 [0.518, 0.586]	+0.091 [+0.045, +0.136]	0.572 [0.524, 0.620]	−0.044 [−0.127, +0.039]
NQ, BM25	2013	0.675 [0.652, 0.698]	0.556 [0.517, 0.594]	+0.119 [+0.076, +0.162]	0.624 [0.581, 0.667]	−0.041 [−0.122, +0.036]
TripClick, dense	1050	0.621 [0.587, 0.655]	0.599 [0.555, 0.642]	+0.022 [−0.028, +0.072]	0.558 [0.497, 0.617]	+0.043 [−0.063, +0.145]
NQ, dense	1932	0.695 [0.669, 0.720]	0.611 [0.583, 0.639]	+0.084 [+0.045, +0.123]	0.512 [0.471, 0.552]	+0.122 [+0.047, +0.197]

BM25 ranking reranked by the same model; relabelling against it (exploratory, `scripts/47`) isolates expansion. The harm AUC becomes 0.583 [0.547, 0.620], and the gap to detection +0.059 [+0.012, +0.107] still excludes zero. Helped-vs.-rest falls to chance (0.504). NQ BM25 needs no correction; the dense runs stored no reranked original, so their labels stay mixed.

Not label sparsity. Most TAIL queries have one relevant document, so failure might track label sparsity. The relevant-document count alone predicts failure at only 0.570 (TripClick) and 0.521 (NQ). Within count strata, detection AUC is 0.644 and 0.677 (TripClick; size-weighted 0.656) and 0.687 and 0.659 (NQ), and the detector stays significant when the count is controlled for.

The gap is largely structural. Expansion rarely rescues a failing query: 84%, 93%, 94%, and 84% of acted-upon failing queries keep nDCG@10 = 0 (ties). Failing queries are therefore mostly "not harmed", so the two labels are coupled, and the harm AUC compares a detector against a label that is itself largely a failure indicator. Dropping the failing queries, where harm is impossible, gives harm AUCs of 0.553, 0.582, 0.627, and 0.655, equal to the full-set value on TripClick BM25 and higher on the other three: the tied failing queries pull the harm AUC down. The dense harm score detects change rather than direction: it ranks *helped* queries as likely harmed (helped-vs.-rest AUC 0.464 and 0.406, against 0.540 and 0.599 under BM25). Restricted to queries whose nDCG the expansion changed (last two columns), the BM25 gaps vanish (point estimates reverse, CIs span zero) because detection itself drops to 0.53–0.58 there; only dense NQ keeps a significant gap (+0.122). Predicting the *direction* of a change is weak everywhere (0.51–0.62). It is better for large effects: on the half of changed queries with the largest $|\Delta|$, it reaches 0.644 and 0.711 under BM25, though only 0.551 and 0.526 under dense retrieval. Under BM25, the action-utility score's errors are therefore concentrated in small changes.

5 Result 2: What Is Each Prediction Worth?

Policies. Fusion and reranking differ between corpora, so we compare policies within a corpus, not across. On TripClick, no policy differs from BM25 or from any other; every pairwise CI spans zero (Table 2). BM25 alone is the strongest unexpanded arm: against BM25 with the same reranker, Static QE gains +0.031 [+0.017, +0.044], a gain that only offsets the reranker's own loss. These nulls are bounded: with MDEs of 0.007–0.011 nDCG@10, effects smaller than that could exist undetected. On NQ, where the expanded ranking is used unfused, unconditional QE significantly harms

Table 2: nDCG@10 gain over BM25 on the full test sets, with 95% CIs and, for real policies, the minimum detectable effect at 80% power (MDE). Retrospective rows use test labels and are upper bounds, not policies. TripClick's Static QE, perfect detector, and oracle all use the RRF-fused ranking; NQ's arms use the unfused expanded ranking.

	TripClick (n=1175)	NQ (n=2052)
BM25 nDCG@10	0.3152	0.3080
Static QE	+0.0007 [−0.007, +0.008]	−0.0217 [−0.032, −0.012]
Failure-selective QE	+0.0027 [−0.002, +0.008]	−0.0067 [−0.014, +0.000]
Closed-loop QE	+0.0009 [−0.007, +0.009]	—
MDE (real policies)	0.007–0.011	0.010–0.015
Perfect detector	+0.0103 [+0.007, +0.014]	+0.0135 [+0.010, +0.017]
Oracle selector	+0.0246 [+0.020, +0.030]	+0.0415 [+0.036, +0.047]

(−0.022). Failure-selective QE recovers most of that loss (+0.015 [+0.008, +0.022] over Static QE); its residual −0.007 against BM25 has a CI that just reaches zero. Failure-selective QE routes 49% (TripClick) and 51% (NQ) of queries to the generator, halving calls relative to Static QE. The closed-loop agent costs 24% more calls (1.22 vs. 0.98 per query; Static QE skips the 1.5% of queries the planner leaves unexpanded) and 36% more latency than Static QE, for no measurable gain.

Headroom decomposition. The oracle selector expands a query iff that helps; it is non-negative by construction and is an upper bound, not a significance result. A *perfect detector* expands exactly the failing queries. Because QE can only help a failing query (TripClick +0.023 on failing vs. −0.018 on healthy queries; NQ +0.028 vs. −0.069), perfect detection is safe, but it recovers only 42% (TripClick) and 33% (NQ) of the oracle's gain: most failing queries are not rescued. The remaining 58% and 67% lies in healthy queries that expansion sometimes helps and sometimes harms, which only an action-utility predictor can separate. Our detector captures part of the first share: its routed queries gain +0.0055 [−0.005, +0.016] on TripClick, against −0.0040 for the rest.

Heterogeneity is real; predicting it depends on the retriever. Expanding the top-k queries ranked by the action-utility score captures at most 12% of the oracle's gain on TripClick (budgets 5–40%, an uplift curve in the style of Qini [15]). Yet the score is not noise. A best-linear-predictor test [4] asks whether it ranks queries by their gain: regressing Δ on the validation-fit score under BM25, the slope is +1.09 [+0.54, +1.67] on TripClick and +1.21 [+0.96, +1.44] on NQ, and the top score quartile's mean Δ exceeds the bottom quartile's by +0.093 and +0.120 (both CIs exclude

Table 3: Detection vs. action utility across interventions (BM25, test sets). LLM QE rows repeat Table 1 (acted-upon queries) and Section 5; reranking and PRF rows use every test query. $\bar{\Delta}$: mean nDCG@10 change when every query receives the intervention (TripClick LLM QE: fused). Harm, Helped: action-utility AUC for harmed-vs.-rest and helped-vs.-rest. Diff.: paired bootstrap of detection minus harm AUC. Share: the perfect detector's gain as a fraction of the oracle selector's. BLP: slope of Δ on the centred action-utility score.

Intervention	Corpus	$\bar{\Delta}$	Detection	Harm	Helped	Diff.	Share	BLP
LLM QE	TripClick	+0.001	0.643	0.552	0.540	+0.091 [+0.045, +0.136]	42%	+1.09 [+0.54, +1.67]
LLM QE	NQ	−0.022	0.675	0.556	0.599	+0.119 [+0.076, +0.162]	33%	+1.21 [+0.96, +1.44]
Reranking	TripClick	−0.028	0.649	0.575	0.513	+0.075 [+0.029, +0.119]	52%	+0.55 [+0.18, +0.93]
Reranking	NQ	+0.225	0.680	0.514	0.679	+0.166 [+0.120, +0.213]	57%	+0.96 [+0.86, +1.06]
PRF	TripClick	−0.001	0.649	0.572	0.393	+0.077 [+0.014, +0.140]	8%	−0.11 [−0.62, +0.39]
PRF	NQ	+0.010	0.680	0.501	0.553	+0.179 [+0.131, +0.227]	32%	+0.10 [−0.23, +0.44]

zero). Under dense retrieval the evidence is weaker: the slope is +0.42 [+0.14, +0.71] on TripClick and +0.11 [−0.01, +0.24] on NQ, not significant. Treatment effects vary across queries, and post-action telemetry tracks them weakly, more so under BM25. The oracle's magnitude is split-sensitive on TripClick (validation +0.053 [0.045, 0.062] vs. test +0.025 [0.020, 0.030]) but not on NQ (+0.040 [0.031, 0.049] vs. +0.042 [0.036, 0.047]), so we read it as an order of magnitude, not a bound to target.

6 Result 3: Beyond LLM QE

Three findings carry over from LLM QE (Table 3).

Detection beats harm prediction in every new setting, by +0.075 to +0.179, and harm prediction is at chance on NQ for both interventions. Predicting help is easier where the intervention helps often: on NQ the reranker helps 50% of queries, and helped-vs.-rest (0.679) ties detection (+0.001 [−0.029, +0.032]); in the other three settings detection beats it too. The action-utility score ranks queries by their gain for reranking (both BLP CIs exclude zero) but not for PRF.

Failure covers part of the headroom. A perfect failure detector recovers 52% and 57% of the oracle's gain for reranking and 8% and 32% for PRF. On TripClick PRF barely moves rankings (82% of queries tied; no validation setting beat BM25), and the little it changes is mostly in healthy queries.

Gating on failure can cost more than it saves. On NQ, reranking every query gains +0.225 [+0.210, +0.240]; reranking only predicted failures keeps +0.114 [+0.103, +0.126], since the reranker also helps healthy queries, while the benefit model reranks 94% of queries and keeps +0.223. On TripClick, reranking every query loses −0.028 [−0.042, −0.013]; the failure gate still loses −0.010 [−0.020, −0.000], and the benefit model, reranking 15% of queries, does not (−0.001 [−0.005, +0.003]). PRF follows the same order on NQ (+0.010 always, +0.004 failure-gated, +0.009 benefit-gated); on TripClick no PRF policy moves nDCG@10 by more than 0.002 (MDE $\leq$ 0.007). These reranking numbers use the current reranker input; TripClick's stored reranked arm (Section 4) predates a fix to that text and scores 0.285 against 0.287 here.

7 Discussion and Limitations

Why action utility is harder. Both outcomes are observed offline, so the difficulty is not counterfactual missingness. Detection describes a ranking that already exists. Action utility depends on what a generator writes from evidence it did not choose; the output is deterministic given its input (greedy decoding) but varies sharply across queries, and its effect on a single relevant document's rank is small and discrete. That fits the evidence above: direction is predictable for large changes and close to chance for small ones. Reranking and PRF involve no generator and show the same gap, so generator variance is not the whole explanation; the small, discrete effect of any intervention on a sparse relevance label is.

Limitations. One small (3B) generator under a two-call budget; a stronger one may change both the rescue rate and the policy results. Reranking and PRF were run under BM25 only, with one cross-encoder and one PRF family. Two corpora; under dense retrieval, retriever strength and domain are confounded. TripClick's relevance labels are click-derived and sparse, which is one reason we report NQ's human labels in parallel. The policy comparisons on NQ use unfused rankings and lack a closed-loop arm. Of the analyses above, two were not pre-committed with the others: the paired difference on changed queries (Table 1, last column) and the reranker-matched relabelling (committed before it was computed, but after review); every pre-committed analysis, including those summarized here in one line, is reported in full in the repository (`docs/revision_analyses_summary.json`).

8 Conclusion

Detecting that retrieval has failed is ordinary QPP; under BM25 our detector is no better than NQC. Knowing that an intervention will fix it is a separate problem, and a failure detector is not a substitute for it. This holds for LLM QE, cross-encoder reranking, and PRF alike: perfect failure detection recovers 8–57% of an oracle selector's headroom, and gating on failure halved the reranker's gain on NQ without stopping its loss on TripClick. The rest requires predicting whether a specific intervention helps a query that is already working, which our telemetry does only weakly, and better for reranking than for expansion. Selective-intervention work should therefore report action utility on the same queries as detection, together with its tie rule, rather than treating a failure detector as a proxy for benefit.

Ethical Considerations

All data used (TripClick, BEIR Natural Questions) are existing research benchmarks, used under their licenses; TripClick is used for non-commercial research, and no model or per-query output fit on it is released. TripClick's click logs are aggregate, de-identified

interactions, and we make no attempt to re-identify searchers. No human-subjects data was collected. The generator is a small, locally hosted model used only to rewrite queries within a bounded action space. Our central finding cautions against deploying generative interventions on the strength of a failure detector alone.

References

[1] Kenya Abe, Kunihiro Takeoka, Makoto P. Kato, and Masafumi Oyamada. 2025. LLM-based Query Expansion Fails for Unfamiliar and Ambiguous Queries. *arXiv preprint arXiv:2505.12694* (2025). https://arxiv.org/abs/2505.12694

[2] Giambattista Amati, Claudio Carpineto, and Giovanni Romano. 2004. Query Difficulty, Robustness, and Selective Application of Query Expansion. In *Advances in Information Retrieval: 26th European Conference on IR Research (ECIR) (LNCS, Vol. 2997)*. 127–137. doi:10.1007/978-3-540-24752-4_10

[3] Eva Ascarza. 2018. Retention Futility: Targeting High-Risk Customers Might Be Ineffective. *Journal of Marketing Research* 55, 1 (2018), 80–98. doi:10.1509/jmr.16.0163

[4] Victor Chernozhukov, Mert Demirer, Esther Duflo, and Iván Fernández-Val. 2018. Generic Machine Learning Inference on Heterogeneous Treatment Effects in Randomized Experiments, with an Application to Immunization in India. *arXiv preprint arXiv:1712.04802* (2018). https://arxiv.org/abs/1712.04802

[5] Kevyn Collins-Thompson. 2009. Reducing the Risk of Query Expansion via Robust Constrained Optimization. In *Proceedings of the 18th ACM Conference on Information and Knowledge Management (CIKM)*. 837–846. doi:10.1145/1645953.1646059

[6] Steve Cronen-Townsend, Yun Zhou, and W. Bruce Croft. 2002. Predicting Query Performance. In *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. 299–306. doi:10.1145/564376.564429

[7] Steve Cronen-Townsend, Yun Zhou, and W. Bruce Croft. 2004. A Framework for Selective Query Expansion. In *Proceedings of the Thirteenth ACM International Conference on Information and Knowledge Management*. 236–237. doi:10.1145/1031171.1031220

[8] Carlos Fernández-Loría and Foster Provost. 2022. Causal Classification: Treatment Effect Estimation vs. Outcome Prediction. *Journal of Machine Learning Research* 23, 59 (2022), 1–35.

[9] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL)*. https://aclanthology.org/2024.naacl-long.389/

[10] Varun Kotte. 2026. Not All Queries Need Rewriting: When Prompt-Only LLM Refinement Helps and Hurts Dense Retrieval. *arXiv preprint arXiv:2603.13301* (2026). https://arxiv.org/abs/2603.13301

[11] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, et al. 2019. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics* 7 (2019), 452–466. doi:10.1162/tacl_a_00276

[12] Victor Lavrenko and W. Bruce Croft. 2001. Relevance-Based Language Models. In *Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. 120–127. doi:10.1145/383952.383972

[13] Yibin Lei, Yu Cao, Tianyi Zhou, Tao Shen, and Andrew Yates. 2024. Corpus-Steered Query Expansion with Large Language Models. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 2: Short Papers)*. 393–401.

[14] Yuanhua Lv and ChengXiang Zhai. 2009. Adaptive Relevance Feedback in Information Retrieval. In *Proceedings of the 18th ACM Conference on Information and Knowledge Management (CIKM)*. 255–264. doi:10.1145/1645953.1645988

[15] Nicholas J. Radcliffe. 2007. Using Control Groups to Target on Predicted Lift: Building and Assessing Uplift Models. *Direct Marketing Analytics Journal* 1 (2007), 14–21.

[16] Navid Rekabsaz, Oleg Lesota, Markus Schedl, Jon Brassey, and Carsten Eickhoff. 2021. TripClick: The Log Files of a Large Health Web Search Engine. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2507–2513. doi:10.1145/3404835.3463242

[17] Sentence Transformers. 2026. cross-encoder/ms-marco-MiniLM-L6-v2. https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2

[18] Anna Shtok, Oren Kurland, David Carmel, Fiana Raiber, and Gad Markovits. 2012. Predicting Query Performance by Query-Drift Estimation. *ACM Transactions on Information Systems* 30, 2 (2012), 11:1–11:35. doi:10.1145/2180868.2180873

[19] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*. https://arxiv.org/abs/2104.08663

[20] Liang Wang, Nan Yang, and Furu Wei. 2023. Query2doc: Query Expansion with Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 9414–9423. doi:10.18653/v1/2023.emnlp-main.585

[21] Orion Weller, Kyle Lo, David Wadden, Dawn Lawrie, Benjamin Van Durme, Arman Cohan, and Luca Soldaini. 2024. When do Generative Query and Document Expansions Fail? A Comprehensive Study Across Methods, Retrievers, and Datasets. In *Findings of the Association for Computational Linguistics: EACL 2024*. 1987–2003.

[22] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. 2023. C-Pack: Packed Resources For General Chinese Embeddings. *arXiv preprint arXiv:2309.07597* (2023). https://arxiv.org/abs/2309.07597

[23] Le Zhang, Yihong Wu, Qian Yang, and Jian-Yun Nie. 2024. Exploring the Best Practices of Query Expansion with Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 1872–1883. doi:10.18653/v1/2024.findings-emnlp.103

[24] Yun Zhou and W. Bruce Croft. 2007. Query Performance Prediction in Web Search Environments. In *Proceedings of the 30th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. 543–550. doi:10.1145/1277741.1277835